Chapitre 7

Régression polynomiale, over-fitting, under-fitting et régularisation

Sommaire

7.1 Modéliser des données par une régression polynomiale 97
7.2 Mise en oeuvre avec `scikit-learn` 99
7.3 Le compromis biais-variance 108
7.4 Techniques de régularisation 111
7.4.1 Régularisation de Ridge 111
7.4.2 Régularisation de Lasso 111
7.4.3 Régularisation ElasticNet 112

7.1 Modéliser des données par une régression polynomiale

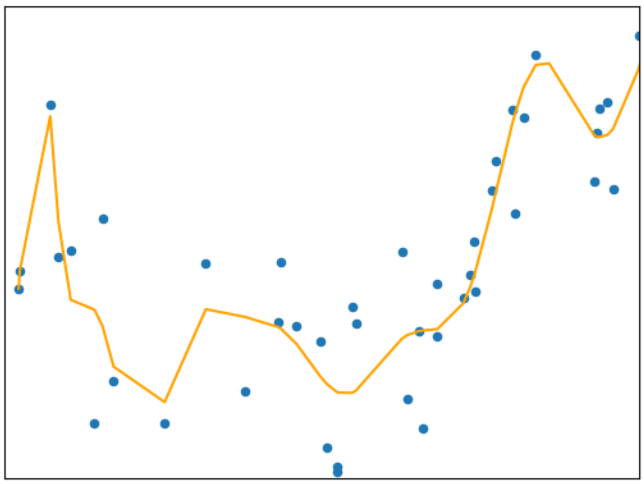
La régression polynomiale est un modèle qui utilise une fonction hypothèse sous la forme d'un polynôme de degré arbitraire, étendant ainsi le modèle de régression linéaire pour lequel la fonction hypothèse est un polynôme de degré 1.

Le choix du degré du polynôme a utiliser à partir d'un jeu arbitraire de données soulève de nouvelles problématiques importantes : l'over-fitting (sur-apprentissage) ou l'under-fitting (sous-apprentissage).Pour répondre à ces problèmes, nous allons introduire le *compromis biais-variance* et les *techniques de régularisation*.

Par ailleurs, après avoir présenter de manière précises les calculs matriciels nécessaires pour implémenter les algorithmes de régression (minimisation de critères de moindres carrés par la méthode de gradient), nous nous intéressons ici à l'utilisation des fonctions existantes dans les bibliothèques Python dédiées au machine learning et notamment `Scikit-learn`.

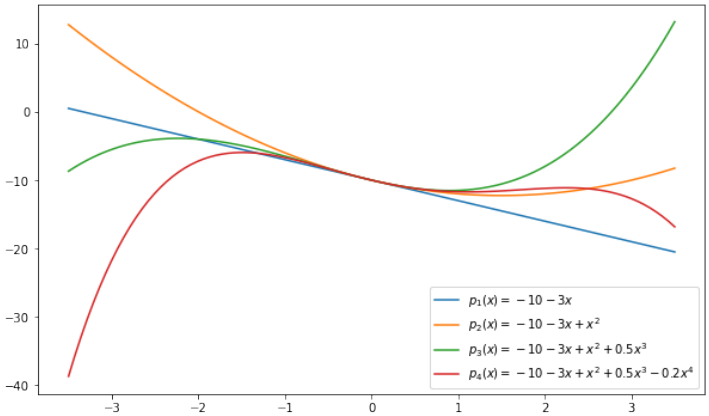
Dans le chapitre précédent, nous avons présenté des modèles de régression linéaire consistant à mettre en évidence des relations de linéarité entre la ou les données d'entrée et la donnée cible. Cela se traduit par une représentation graphique de ces données à partir desquelles il est possible de tracer une droite (modèle univarié) ou encore un plan (modèle multivarié à deux variables explicatives) de régression.

Si maintenant, nous disposons de données qui ne présentent pas cette propriété de corrélation linéaire, il faut alors trouver d'autres modèles de régression ou de fonctions hypothèses non linéaires, comme illustrées par la figure suivante.



Le propos de ce chapitre consiste ainsi à rechercher des fonctions hypothèses sous la forme de polynômes. Nous représentons ci-dessous sur l'intervalle [-3, 3], trois polynômes de degré respectif 1, 2 et 3 permettant ainsi d'enrichir les techniques de régression lorsqu'il n'y a manifestement pas de corrélation linéaire entre les entrées et la cible des données étudiées.

[2]:
```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-3.5,3.5,100)
y1 = -10-3*x
y2 = -10-3*x+x**2
y3 = -10-3*x+x**2+0.5*x**3
y4 = -10-3*x+x**2+0.5*x**3-0.2*x**4
fig = plt.figure(1, figsize=(10, 6))
plt.plot(x,y1, label="$p_1(x) = -10-3x$")
plt.plot(x,y2, label="$p_2(x) = -10-3x+x^2$")
plt.plot(x,y3, label="$p_3(x) = -10-3x+x^2+0.5x^3$")
plt.plot(x,y4, label="$p_4(x) = -10-3x+x^2+0.5x^3-0.2x^4$")
plt.legend()
plt.show()
```



Ainsi nous allons pouvoir formuler à partir d'un jeu de données (X,y) des fonctions hypothèses de la forme suivante :

— si X n'est composé que d'une seule variable explicative, notée X_1 (régression univariée), la fonction hypothèse à base d'un polynôme de degré k s'écrit de la manière suivante :

$$h(X) = \theta_0 + \Theta_1 X_1 + \Theta_2 X_1^2 + \ldots + \Theta_k X_1^k + \cdots = \theta_0 + \sum_{j=1}^{k} \theta_j X_1^j$$

— si X est composé de plusieurs variables explicatives (régression multivariée), par exemple ici X_1 et X_2, la fonction hypothèse à base d'un polynôme de degré k s'écrit de la manière suivante :

$$h(X) = \theta_0 + \sum_{j=1}^{k} \left(\theta_{1,j} X_1^j + \theta_{2,j} X_2^j \right)$$

En pratique, les calculs de minimisation du critère de moindres carrés par la méthode de gradient s'opèrent de la même manière que dans le cas de la régression linéaire univariée ou multivariée (qui correspond à $k = 1$). La généralisation des formules vues dans le cas linéaire est sans difficulté. Elle n'est pas présentée ici et faitt l'objet d'un exercice de TP.

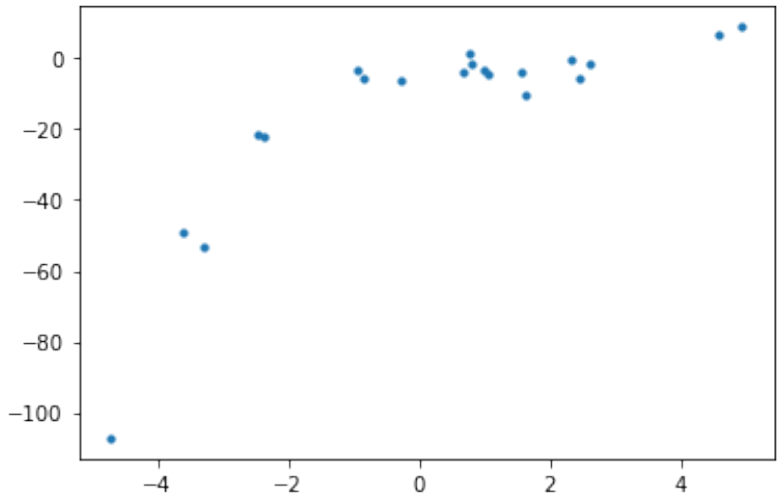
Dans les implémentations qui suivent, nous ne développons donc pas ces formules de calcul mais nous utilisons les fonctions implémentées dans la bibliothèque `scikit-learn` dédiée au maching learning.

7.2 Mise en oeuvre avec `scikit-learn`

Nous commençons par générer un jeu de données généré avec un polynome auquel est ajouté un "bruit" aléatoire.

[8]:
```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
x = 2 - 3 * np.random.normal(0, 1, 20)
y = x - 2 * (x ** 2) + 0.5 * (x ** 3) + np.random.normal(-3, 3, 20)
plt.scatter(x,y, s=10) # s pour size/taille des points
plt.show()
```



Nous implémentons ci-dessous un calcul de régression linéaire sur le jeu de données précédent, en utilisant la bibliothèque *scikit-learn (sklearn)*.

[4]:
```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(0)
x = 2 - 3 * np.random.normal(0, 1, 20)
y = x - 2 * (x ** 2) + 0.5 * (x ** 3) + np.random.normal(-3, 3, 20)
print("x.shape : ",x.shape) # sert à comprendre la transformation à
 ↪effectuer

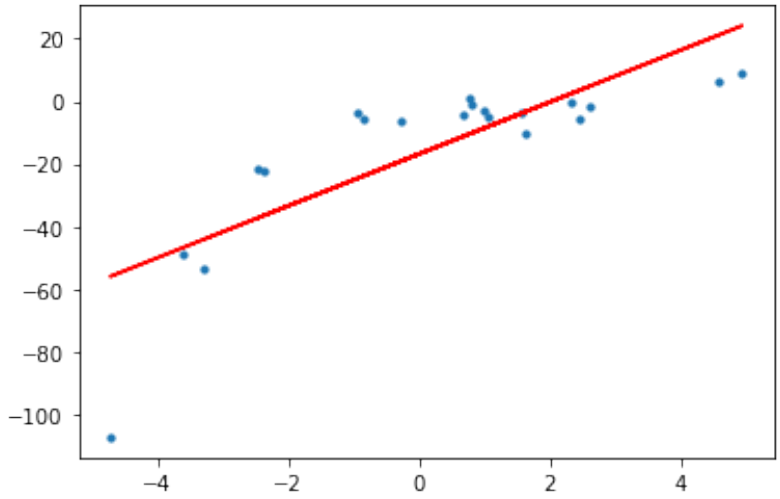
# transformation des vecteurs 1D de format (n,) en une matrice 2D de n
 ↪lignes et 1 colonne, de format (n,1)
x = x[:, np.newaxis]
y = y[:, np.newaxis]
print("x.shape : ",x.shape) # sert à comprendre la transformation
 ↪effectuée

model = LinearRegression()
model.fit(x, y)
y_pred = model.predict(x)

plt.scatter(x, y, s=10)
plt.plot(x, y_pred, color='r')
plt.show()

rmse = np.sqrt(mean_squared_error(y,y_pred))
r2 = r2_score(y,y_pred)
print("rmse =", rmse, ", r2 =", r2)
```

```
x.shape :  (20,)
x.shape :  (20, 1)
```




```
rmse = 15.908242501429998 , r2 = 0.6386750054827146
```

Le programme se termine par le calcul de 2 indicateurs :

— `rmse` - Root of Mean Squared Error - correspond à la distance entre les valeurs prédites, $\{h(x_i), 0 \leq i \leq n\}$ et les valeurs des données $\{y_i, 0 \leq i \leq n\}$;

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(h(x_i)-y_i)^2}$$

Cet indicateur mesure ainsi la proximité de la courbe de régression (de la fonction hypothèse) avec les données.

— `r2` ou `r2_score`, appelé *coefficient de détermination* va permettre d'évaluer la qualité de la prédiction :

$$1-\frac{\sum_{i=1}^{n}(h(x_i)-y_i)^2}{\sum_{i=1}^{n}(\overline{y}-y_i)^2}$$

où $\overline{y}=\frac{1}{n}\sum_{i=1}^{n}y_i$ est un calcul de moyenne. La valeur numérique de `r2` est comprise entre 0 et 1. Si elle vaut 1, la régression est parfaite par rapport au jeu de données. Si elle vaut 0, la régression n'est pas du tout pertinente pour le jeu de données.

Nous allons maintenant définir une fonction hypothèse de régression à partir d'un polynome d'ordre k :

$$h(x)=\theta_0+\sum_{j=1}^{k}\theta_j x^j$$

Cette expression reste linéaire par rapport aux θ_j, les coefficients à calculer.

Pour simplifier les calculs qui suivent (sans perte de généralité), nous recherchons un modèle/hypothèse univarié de regression polynomiale uniquement de de degré 3, à partir des données (x_i, y_i) :

$$h(x)=\theta_0+\theta_1 x+\theta_2 x^2+\theta_3 x^3$$

Calcul de régression basé sur `sckikit-learn`

Nous commençons alors par déclarer le degré du modèle :

```
poly = PolynomialFeatures(degree=3)
```

Nous réécrivons le modèle précédent sous la forme **linéaire** suivante :

$$h(t,u,v,w)=\theta_0 t+\theta_1 u+\theta_2 v+\theta_3 w$$

avec $t=1$, $u=x$, $v=x^2$ et $w=x^3$

Nous venons ainsi de transformer un modèle polynomial de degré 3, utilisant une variable explicative (x) en un modèle linéaire (de degré 1) utilisant quatre variables explicatives (t,u,v,w).

Cette transformation est effectuée dans `sckikit-learn` grâce à la fonction

```
x_poly = poly.fit_transform(x)
```

qui construit une matrice de données telle que la ligne i corresponde à (t_i, u_i, v_i, w_i), c'est à dire $(1, x_i, x_i^2, x_i^3)$.

Il ne reste plus qu'à appeler un calcul de régression linéaire sur ce nouvel espace de données, comme cela est décrit dans le script de calcul de régression linéaire présenté précédemment.

[5]:
```
import operator

import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(0)
x = 2 - 3 * np.random.normal(0, 1, 20)
x.sort()
y = x - 2 * (x ** 2) + 0.5 * (x ** 3) + np.random.normal(-3, 3, 20)

# transformation des vecteurs 1D en matrice à 1 colonne
x = x[:, np.newaxis]
y = y[:, np.newaxis]
print("x =\n",x)

#version interactive du choix du polynome
print("Donner le degré de régression polynomiale souhaité : ")
ndegre=int(input())

polynomial_features= PolynomialFeatures(degree=ndegre)
x_poly = polynomial_features.fit_transform(x)
print("x_poly =\n",x_poly)

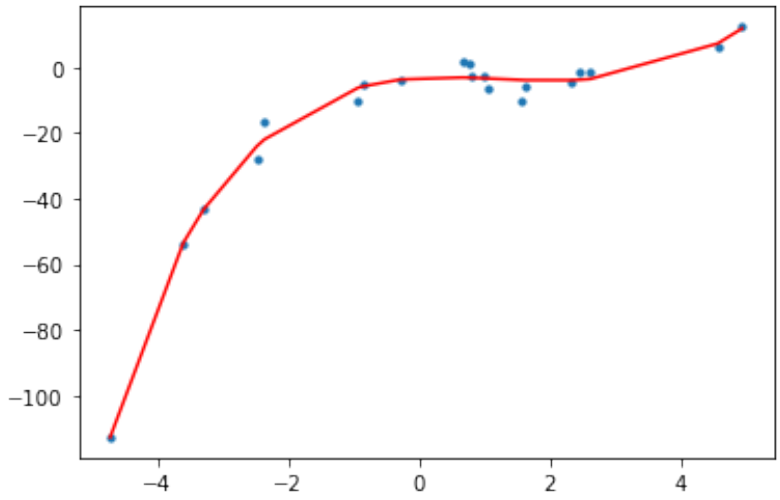
model = LinearRegression()
model.fit(x_poly, y)
y_poly_pred = model.predict(x_poly)

rmse = np.sqrt(mean_squared_error(y,y_poly_pred))
r2 = r2_score(y,y_poly_pred)
print("rmse =", rmse, ", r2 =", r2)

plt.scatter(x, y, s=10)
plt.plot(x, y_poly_pred, color='r')
plt.show()
```

```
x =
 [[-4.7226796 ]
 [-3.60267397]
 [-3.29215704]
 [-2.48223722]
 [-2.36282052]
 [-0.93621395]
 [-0.85026525]
 [-0.28311318]
 [ 0.6684103 ]
 [ 0.76820449]
 [ 0.79952837]
 [ 0.99897702]
 [ 1.0607969 ]
 [ 1.56786929]
 [ 1.63497495]
 [ 2.30965656]
 [ 2.45407162]
```

```
 [ 2.61547479]
 [ 4.56228722]
 [ 4.93183364]]
Donner le degré de régression polynomiale souhaité :
7
x_poly =
 [[ 1.00000000e+00 -4.72267960e+00  2.23037026e+01 -1.05333241e+02
   4.97455149e+02 -2.34932128e+03  1.10950917e+04 -5.23985631e+04]
 [ 1.00000000e+00 -3.60267397e+00  1.29792597e+01 -4.67600412e+01
   1.68461183e+02 -6.06910720e+02  2.18650145e+03 -7.87725188e+03]
 [ 1.00000000e+00 -3.29215704e+00  1.08382980e+01 -3.56813789e+01
   1.17468703e+02 -3.86725416e+02  1.27316080e+03 -4.19144529e+03]
 [ 1.00000000e+00 -2.48223722e+00  6.16150161e+00 -1.52943086e+01
   3.79641021e+01 -9.42359073e+01  2.33915877e+02 -5.80634695e+02]
 [ 1.00000000e+00 -2.36282052e+00  5.58292081e+00 -1.31914399e+01
   3.11690048e+01 -7.36467642e+01  1.74014086e+02 -4.11164053e+02]
 [ 1.00000000e+00 -9.36213952e-01  8.76496565e-01 -8.20588313e-01
   7.68246228e-01 -7.19242837e-01  6.73365179e-01 -6.30413876e-01]
 [ 1.00000000e+00 -8.50265253e-01  7.22951000e-01 -6.14700114e-01
   5.22658148e-01 -4.44398062e-01  3.77856231e-01 -3.21278023e-01]
 [ 1.00000000e+00 -2.83113175e-01  8.01530701e-02 -2.26923902e-02
   6.42451465e-03 -1.81886474e-03  5.14944573e-04 -1.45787593e-04]
 [ 1.00000000e+00  6.68410302e-01  4.46772332e-01  2.98627229e-01
   1.99605516e-01  1.33418383e-01  8.91782219e-02  5.96076422e-02]
 [ 1.00000000e+00  7.68204494e-01  5.90138145e-01  4.53346775e-01
   3.48263030e-01  2.67537225e-01  2.05523298e-01  1.57883922e-01]
 [ 1.00000000e+00  7.99528375e-01  6.39245622e-01  5.11095014e-01
   4.08634966e-01  3.26715250e-01  2.61218113e-01  2.08851293e-01]
 [ 1.00000000e+00  9.98977018e-01  9.97955082e-01  9.96934192e-01
   9.95914346e-01  9.94895544e-01  9.93877783e-01  9.92861064e-01]
 [ 1.00000000e+00  1.06079690e+00  1.12529005e+00  1.19370419e+00
   1.26627770e+00  1.34326345e+00  1.42492970e+00  1.51156100e+00]
 [ 1.00000000e+00  1.56786929e+00  2.45821410e+00  3.85415839e+00
   6.04281656e+00  9.47434649e+00  1.48545369e+01  2.32899721e+01]
 [ 1.00000000e+00  1.63497495e+00  2.67314309e+00  4.37052199e+00
   7.14569397e+00  1.16830307e+01  1.91014625e+01  3.12304126e+01]
 [ 1.00000000e+00  2.30965656e+00  5.33451340e+00  1.23208939e+01
   2.84570333e+01  6.57259734e+01  1.51804425e+02  3.50616086e+02]
 [ 1.00000000e+00  2.45407162e+00  6.02246754e+00  1.47795667e+01
   3.62701153e+01  8.90094607e+01  2.18435592e+02  5.36056588e+02]
 [ 1.00000000e+00  2.61547479e+00  6.84070838e+00  1.78917003e+01
   4.67952912e+01  1.22391904e+02  3.20112941e+02  8.37247327e+02]
 [ 1.00000000e+00  4.56228722e+00  2.08144647e+01  9.49615661e+01
   4.33241939e+02  1.97657416e+03  9.01769903e+03  4.11413330e+04]
 [ 1.00000000e+00  4.93183364e+00  2.43229830e+01  1.19956906e+02
   5.91607504e+02  2.91770979e+03  1.43896593e+04  7.09674058e+04]]
rmse = 2.90949761111065 , r2 = 0.9886685606325732
```



Dans le script suivant, nous reprenons le même calcul en faisant varier le degré du polynôme de régression parmi les valeurs : $\{1, 2, 3, 20\}$

[6]:
```
import operator

import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(0)
x = 2 - 3 * np.random.normal(0, 1, 20)
x.sort()
y = x - 2 * (x ** 2) + 0.5 * (x ** 3) + np.random.normal(-3, 3, 20)

# transformation des vecteurs 1D en matrice à 1 colonne
x = x[:, np.newaxis]
y = y[:, np.newaxis]

fig = plt.figure(1, figsize=(14, 8))

plt.scatter(x, y, s=10)

for ndegre in [1,2,3,20]:
    polynomial_features= PolynomialFeatures(degree=ndegre)
    x_poly = polynomial_features.fit_transform(x)

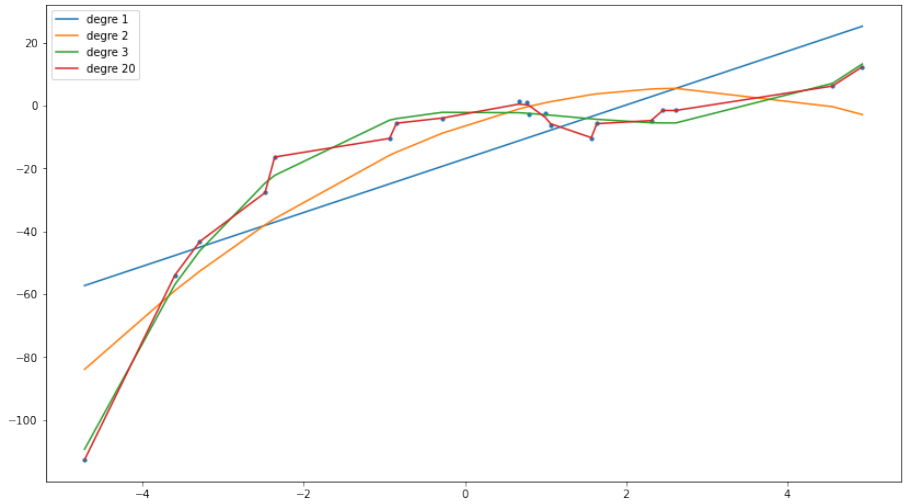
    model = LinearRegression()
    model.fit(x_poly, y)
    y_poly_pred = model.predict(x_poly)

    #rmse = np.sqrt(mean_squared_error(y,y_poly_pred))
    #r2 = r2_score(y,y_poly_pred)
    #print("rmse =", rmse, ", r2 =", r2)

    plt.plot(x, y_poly_pred, label="degre "+str(ndegre))

plt.legend()
```

```
plt.show()
```



Après avoir comparer les courbes polynomiales de régression sur une même figure, nous les traçons dans le script suivant, les unes après les autres, en affichant pour chaque courbe et donc pour chaque degré de polynôme, les indicateurs `rmse` et `r2`.

[7]:
```
import operator

import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import PolynomialFeatures

np.random.seed(0)
x = 2 - 3 * np.random.normal(0, 1, 20)
x.sort()
y = x - 2 * (x ** 2) + 0.5 * (x ** 3) + np.random.normal(-3, 3, 20)

# transformation des vecteurs 1D en matrice à 1 colonne
x = x[:, np.newaxis]
y = y[:, np.newaxis]

fig, g = plt.subplots(6, figsize=(8, 24))
ig = 0

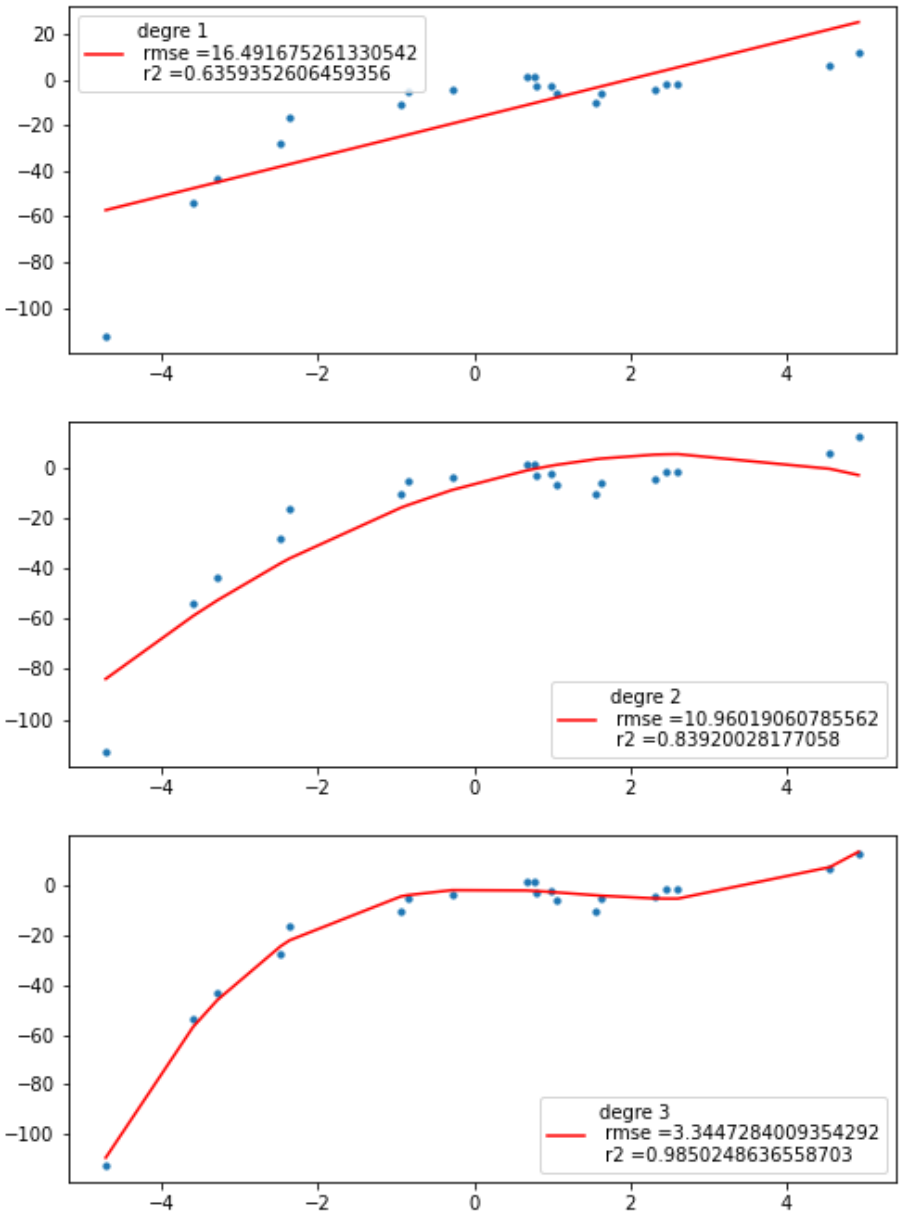
for ndegre in [1,2,3,6,10,20]:
    g[ig].scatter(x,y,s=10)
    polynomial_features= PolynomialFeatures(degree=ndegre)
    x_poly = polynomial_features.fit_transform(x)

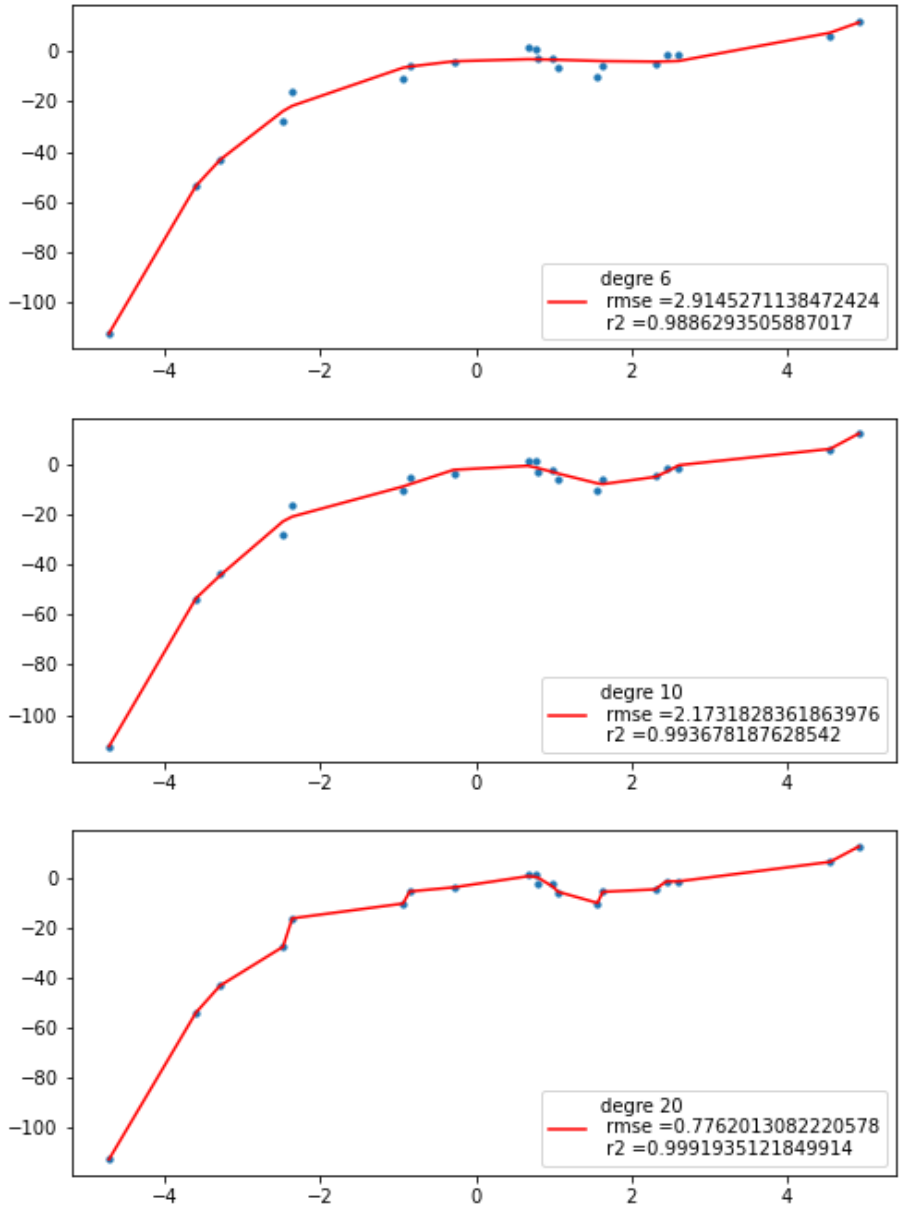
    model = LinearRegression()
    model.fit(x_poly, y)
    y_poly_pred = model.predict(x_poly)

    rmse = np.sqrt(mean_squared_error(y,y_poly_pred))
    r2 = r2_score(y,y_poly_pred)
```

```
    g[ig].plot(x, y_poly_pred, c='r', label="degre "+str(ndegre)+"\n rmse
↪="+str(rmse)+"\n r2 ="+str(r2))
    g[ig].legend()
    ig = ig+1

#plt.show()
```





La régression polynomiale de degré 20 fait apparaître des irrégularités qui correspondent au "bruit" qui est présent dans les données. On parle alors de phénomène de **sur-apprentissage (over-fitting)**. La courbe est très proche des données mais si on ajoutait des données supplémentaires, la courbe du polynôme de degré 20 pourrait être assez différente. Ses oscillations sont instables par rapport à l'ajout de points.

A l'opposé, la régression de degré 1 passe assez loin des données car la complexité du modèle (hypothèse) est trop faible. On parle alors de **sous-apprentissage (under-fitting)**. L'ajout d'une donnée va très peu modifié l'allure de la courbe de régression qui est stable par rapport à l'ajout de points.

Ces deux situations de sous-apprentissage et de sur-apprentissage ne sont pas spécifiques au problème de régresssion polynomiale mais peuvent se rencontrer dans de nombreux modèles/hypothèses qui sont paramètrables.

Le phénomène de sur-apprentissage est un problème difficile en science des données, puisqu'il faudrait savoir identifier ou mesurer le bruit présent dans les données. La question est donc de savoir choisir

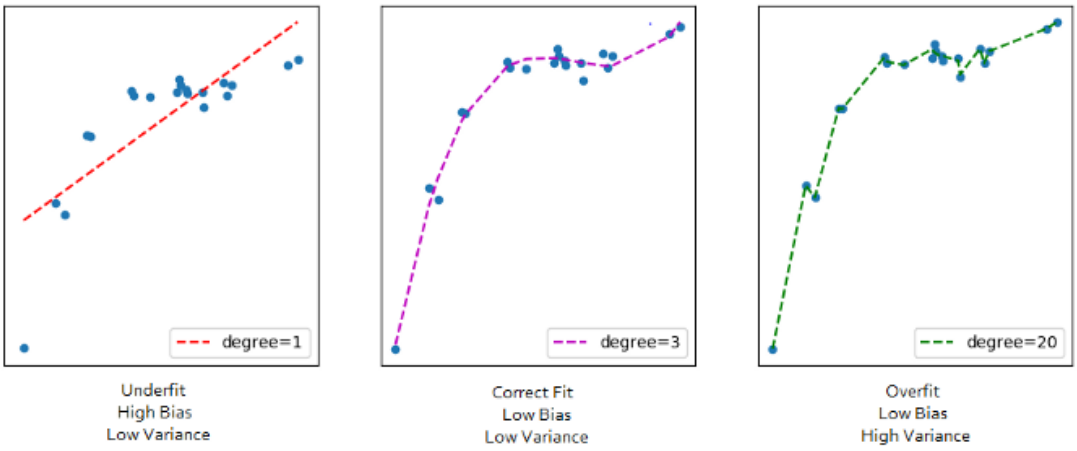
une *complexité* de modèle pertinente, ici le degré "optimal" de régression polynomiale. La notion de **compromis biais-variance** vise à apporter des méthodes d'analyse pour identifier ce degré optimal ou plus généralement le réglage de la complexité dans des modèles/hypothèses.

7.3 Le compromis biais-variance

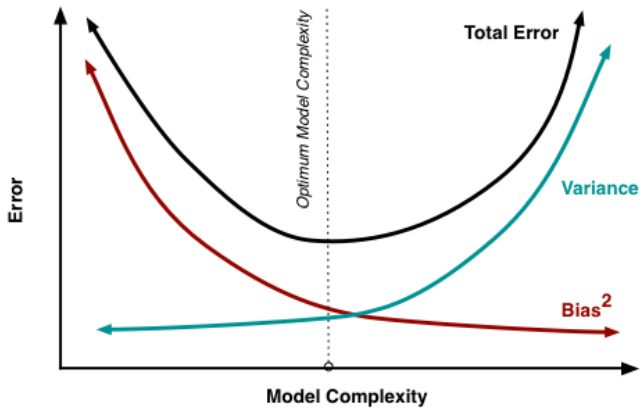
Le biais correspond à l'écart entre les données et le modèle. Si sa valeur est élevée, cela signifie que l'on est dans la situation de sous-apprentissage. les valeurs de `rmse` ou de `r2` permettent de mesurer le biais.

La variance correspond à une mesure du phénomène de sur-apprentissage dgénéré parl'usage d'un modèle *trop complexe*, c'est à dire trop sensible à des petites modifications apportées sur les données.

On illustre ces valeurs du biais et de la variance sur la figure suivante :



Nous pouvons observer sur ces figures que le biais va décroître avec le degré du polynôme choisi alors que la variance va croître. Le *compromis* consiste donc à trouver une valeur *optimale* du degré pour lequel à la fois le biais et la variance ne sont pas trop grands, comme illustré sur la figure suivante où la courbe appelée **erreur totale** est une somme du biais et de la variance. Elle est parfois qualifiée d'**erreur de prédiction** [3] pour bien la distinguer des calculs d'erreurs de type `rmse`.



Définition mathématique

Soit un jeu de données $\{(x_i, y_i); 1 \leq i \leq n\}$, où (x_i) sont les données d'entrée du modèle (chaque x_i est, soit une valeur simple dans un problème univarié, soit un vecteur de plusieurs valeurs dans un problème multivarié) et (y_i) sont les données cibles. On note $h(x)$, la fonction hypothèse.

Comme dans la construction effectuée en début de chapitre, nous supposons que les données sont *bruitées* et le calcul de régression consiste alors à rechercher une fonction que l'on note $f(x)$ qui lisse ce bruit et que l'on définit à partir de l'expression :

$$y_i = f(x_i) + \epsilon_i$$

où ϵ_i est un bruit dont la moyenne $\mu(\epsilon)$ est nulle et l'écart type est notée σ^2.

Rappels :
Définitions de la moyenne et de la variance d'une suite de données $z = (z_i, 1 \leq i \leq n)$

— Calcul de la moyenne

$$\mu(z) = \frac{1}{n} \sum_{i=1}^{n} z_i$$

— La variance notée est la moyenne des carrés des écarts à la moyenne

$$V(z) = \frac{1}{n} \sum_{i=1}^{n} (z_i - \mu(z))^2$$

On peut montrer que

$$V(z) = \left(\frac{1}{n} \sum_{i=1}^{n} z_i^2 \right) - \mu(z)^2 = \mu(z^2) - \mu(z)^2$$

où on note $z^2 = (z_i^2, 1 \leq i \leq n)$

— L'écart-type est la racine carrée de la variance

$$\sigma = \sqrt{V(z)}$$

Il permet de mesurer la dispersion des valeurs z_i autour de la moyenne $\mu(z)$.

A partir du jeu de données $\{(x_i, y_i); 1 \leq i \leq n\}$, nous appelons échantillons d'apprentissage un ensemble S de m échantillons correspondant à des extractions des données initiales dont les tailles sont inférieures à n. Pour chaque échantillon de données, nous notons x^p, l'échantillon p des données d'entrée (attention p est un indice supérieur et non une élévation à la puissance).

Nous effectuons alors un calcul de régression à partir de chaque échantillon pour trouver les coefficients des fonctions hypothèses associées à l'échantillon. La fonction hypothèse associée à l'échantillon x^p est notée :

$$h_{x^p}(x))$$

Pour une valeur donnée de x, chaque fonction hypothèse donne une valeur différentes en x car chacune est associé à un échantillon différents qui a servi à mener le calcul de régression qui permet de calculer les paramètres caractéristiques de la fonction hypothèse.

On définit alors la fonction $G(x)$ qui calcule la moyenne des valeurs renvoyées par toutes les fonctions hypothèses associées à un enchantillonage S :

$$G(x) = \frac{1}{m} \sum_{x^p \in S} h_{x^p}(x)$$

où m est le nombre d'échantillons x_p dans S.

Nous appelons **espérance de h par rapport à l'échantillonage** S sur les données $X = x_i, 1 \leq i \leq n$, la moyenne des valeurs que prend la fonction G sur tous les x_i :

$$E_S[h(X)] = \frac{1}{n} \sum_{i=1}^{n} G(x_i)$$

Nous pouvons maintenant définir de manière plus formelle les deux composantes du compromis biais-variance, relativement à un échantillonage S :

— le biais se calcule par

$$B(h(X)) = E_S[(h - f)(X)]$$

— la variance se calcule par

$$Var(h(X)) = E_S[h^2(X)] - (E_S[h(X)])^2$$

On définit ensuite l'**erreur de prédiction** par

$$Err(h(X)) = B^2(h(X)) + Var(h(X)) + \sigma^2$$

où σ^2 est l'écart-type sur les données bruitées.

La fonction $f(x)$ n'étant pas une fonction connue, nous ne pouvons pas calculer directement le biais qui mesure l'écart entre l'hypothèse et les données *lissées* par $f(x)$. Nous verrons en TP comment il est possible de faire une estimation simple mais fiable du biais.

Le calcul de la variance n'utilisant pas $f(x)$, nous pouvons par contre l'évaluer, à condition de mettre en place un échantillonnage S. Elle mesure la dispersion des fonctions hypothèses liées à un échantillon à la moyenne de ces fonctions hypothèses pour tous les échantillons. En cas de sur-apprentissage, cette dispersion peut être grande comme cela a été expliqué dans les exemples précédents. Nous verrons en TP comment il est possible de faire une estimation simple de la variance.

Le calcul du compromis biais-variance reste assez technique si nous respectons sa définition théorique. Il nécessite l'extraction d'un échantillonage. Nous verrons en TP comment il est possible de l'évaluer simplement pour finalement obtenir une information fiable qui répond à nos attentes, à savoir estimer une *bonne* valeur de la complexité à retenir pour construire le modèle, c'est à dire la fonction hypothèse. Cette complexité correspond ici, au choix du degré du polynôme dans une régression polynomiale, en fonction des données du problème.

Référence sur le compromis biais-variance

1. Dilemme biais-variance : article de Wikipedia
2. Scott Fortmann-Roe. "Understanding the Bias-Variance Tradeoff", blog en ligne, 2012
3. T. Hasyie, R. Tibshirani, J. Friedman. "The Elements of Statistical Learning", Springer, 2017 (2nd edition)
4. G. Gigerenzer, H. Brighton (2009) "Homo Heuristicus : Why biased Minds make Better Inferences", Topics in Cognitive Science 1, 107-143. Accessible en ligne ici

7.4 Techniques de régularisation

Dans les méthodes de régression, la fonction de coût $J(\theta)$ sert à ajuster les paramètres du modèle / fonction hypothèse en fonction des données.

Avec la régression polynomiale, des problèmes de sur-apprentissage peuvent apparaître et alors la minimisation de cette fonction de coût produit des modèles *instables*et donc tr§s sensibles à des petites modifications sur les données.

Pour pallier à ce problème, le principe de la régularisation est de contraindre l'espace de recherche des paramètres θ en modifiant la fonction coût : $J(\theta)$. Il s'agit donc de *pénaliser* certaines valeurs que pourrait prendre θ en empêchant que celles-ci conduisent au minimum de la fonction de coût. On parle alors de *rétressissement* de l'espace de recherche d'où le terme *shrinkage* en anglais.

Pratiquement, nous modifions la fonction de coût en ajoutant une *fonction de pénalité* $P(\lambda, \theta)$ qui dépend de θ et d'un paramètre λ à ajuster afin de produire l'effet souhaité de régularisation :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2 + P(\lambda, \theta)$$

où $\{(x_i, y_i); 1 \leq i \leq m\}$ sont les données et $(\theta_j; 0 \leq j \leq n)$ les paramètres de la fonction hypothèse à ajuster.

Nous présentons dans la suite les techniques de régularisation les plus fréquentes.

7.4.1 Régularisation de Ridge

La fonction de pénalisation se base sur l'utilisation de la norme L_2 du vecteur θ. Appelée aussi norme euclidienne, elle correspond aux métriques et notamment aux distances usuelles en géométrie. La fonction de pénalisation s'écrit :

$$P_r(\lambda, \theta) = \lambda \sum_{j=0}^{n} \theta_j^2$$

et donc la fonction coût "pénalisée" s'écrit

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2 + \lambda \sum_{j=0}^{n} \theta_j^2$$

Dans cette expression, des grandes valeurs de θ_j vont *pénaliser* le calcul de la recherche du minimum. Elles seront ainsi exclues des solutions potentielles.

7.4.2 Régularisation de Lasso

La fonction de pénalisation lasso (Least Absolute Shrinkage and Selection Operator) est identique mais se base sur la norme L_1 :

$$P_l(\lambda, \theta) = \lambda \sum_{j=0}^{n} |\theta_j|$$

7.4.3 Régularisation ElasticNet

La fonction de pénalisation est une composition des deux précédentes, en ajoutant un nouveau paramètre α mesurant l'importance donnée à l'une des deux fonctions de pénalisation précédente par rapport à l'autre.

$$P_e(\lambda, \theta) = \lambda \sum_{j=0}^{n} \left(\alpha\theta_j^2 + (1-\alpha)|\theta_j|\right)$$

L'impact des techniques de régularisation avec ces fonctions de pénalisation sera étudié dans les TP.

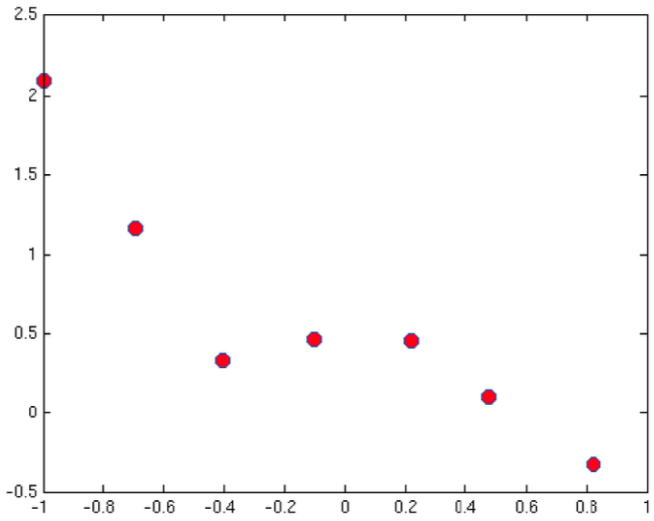
Références sur les techniques de régularisation

1. E. Biernat et M. Lutz (2015) "Data science : fondamentaux et études de cas : Machine learning avec Python et R", Editions Eyrolles
2. Blog de Xavier Dupré, Régression Ridge, Lasso et nouvel estimateur

TP Régression polynomiale et régularisation

Données du problème

Télécharger l'archive correspondant aux données de l'exercice 3/TP 3, décompactez-la. cette archive contient deux fichiers, un pour les données d'entrée x qui sont ici à une dimension, et l'autre pour les données de sorties y à une dimension également. Tracer un graphique représentant le nuage de points défini par ces données dans le style du celui qui est représenté dans la figure suivante.



Analyse du problème

On propose de modéliser ces données par une fonction hypothèse polynomiale de degré 5 :

$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + +\theta_3 x^3 + \theta_4 x^4 + \theta_5 x^5$$

Comme expliqué dans le cours, cela revient à considérer 6 descripteurs constituant les différentes puissances de x dans le polynôme. Le problème reste cependant linéaire par rapport à ces descripteurs.

Le degré du polynôme étant 5 et le nombre de points initiaux 7, le risque de sur-apprentissage (overfitting) est grand. Il est donc nécessaire de *régulariser* le modèle.

Pour cela on va corriger le calcul de la fonction de coût en utilisant une variante de la régularisation de Ridge vu en cours et qui s'écrit :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2 + \lambda \sum_{j=1}^{n} \theta_j^2$$

Le paramètre de régularisation λ va permettre d'éviter des valeurs importantes de θ qui feraient augmenter la valeur de la fonction de coût. A noter que la composantes θ_0 n'intervient pas dans la sommation devant λ.

Modification des formulations matricielles pour intégrer le terme de régularisation de type Ridge

L'introduction de la régularisation intervient dans la fonction de calcul du coût et dans le calcul de la minimisation de la fonction coût avec la méthode de gradient

Calcul du coût

Formule du coût avec la régularisation proposée dans l'exercice (variante de la régularisation de Ridge) :

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2 + \lambda \sum_{j=1}^{n} \theta_j^2$$

Comme vu précédemment, la première partie de cette formule (avant le terme de régularisation)

$$\frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2$$

s'écrit en notation matricielle

$$\frac{1}{2m} \left[h_\theta(X) - Y\right]^t \left[h_\theta(X) - Y\right]$$

avec

$$\text{avec} \quad h_\theta(X) = X\theta \tag{7.1}$$

$$= \begin{bmatrix} 1 & x_{11} & \cdots & x_{1n} \\ \vdots & \vdots & & \vdots \\ 1 & x_{m1} & \cdots & x_{mn} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_n \end{bmatrix} \tag{7.2}$$

$$\text{et} \quad Y = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix} \tag{7.3}$$

On doit alors ajouter à la fonction coût, le terme $\lambda \sum_{j=1}^{n} \theta_j^2$ (attention : on a exclu θ_0). En notation matricielle, cette expression s'écrit

$$\lambda[\theta_-]^t[\theta_-] \quad \text{où} \quad \theta_- = \begin{bmatrix} 0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

Donc au final $J(\theta)$ s'écrit en notation matricielle

$$J(\theta) = \frac{1}{2m} \left[X\theta - Y\right]^t \left[X\theta - Y\right] + \lambda[\theta_-]^t[\theta_-]$$

Calcul de la minimisation de la fonction de coût régularisée avec la méthode de gradient

A chaque itération, on calcule une nouvelle valeur de θ^* à partir de θ telle que $J(\theta^*) < J(\theta)$.

La méthode de gradient correspond au calcul suivant :

$$\theta^* = \theta - \alpha \nabla J(\theta)$$

$$\begin{cases} \theta_0^* & = \theta_0 - \alpha \dfrac{\partial}{\partial \theta_0} J(\theta) \\ \vdots & \\ \theta_k^* & = \theta_k - \alpha \dfrac{\partial}{\partial \theta_k} J(\theta) \\ \vdots & \\ \theta_k^* & = \theta_k - \alpha \dfrac{\partial}{\partial \theta_k} J(\theta) \end{cases}$$

$$\frac{\partial}{\partial \theta_k} J(\theta) = \frac{\partial}{\partial \theta_k} \left(\frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x_i) - y_i \right)^2 + \lambda \sum_{j=1}^{n} \theta_j^2 \right) \tag{7.4}$$

$$= \frac{1}{m} \left(\sum_{i=1}^{m} \left(h_\theta(x_i) - y_i \right) x_{ik} \right) + 2\lambda\theta_k \tag{7.5}$$

Le dernier terme du membre droit de l'équation $2\lambda\theta_k$ vaut 0 si $k = 0$ en raison de la formule de régularisation utilisée.

La formulation matricielle s'écrit ainsi (en utilisant que $h_\theta(X) = X\theta$) :

$$\theta^* = \theta - \alpha \left[\frac{1}{m} X^t [X\theta - Y] + 2\lambda\theta_- \right]$$

Discussion sur le choix de la régularisation de type Ridge utilisée

Afin de comprendre l'intérêt du choix de la régularisation utilisée ici, à savoir une régularisation Ridge sans prendre le premier terme θ_0, on pourra comparer les résultats que l'on obtiendra dans la mise en oeuvre proposée ci-dessous avec une régularisation de Ridge complète et intégrant θ_0.

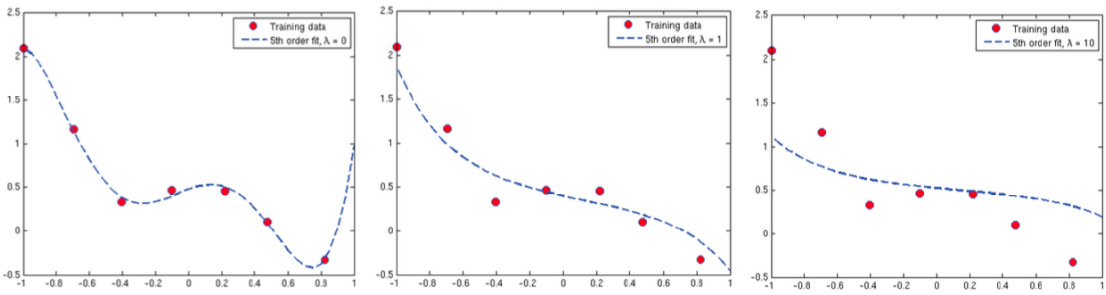
Mise en oeuvre

Il s'agit d'adapter l'algorithme de régression linéaire écrite sous forme matricielle et modifié à partir des formules précédents pour intégrer le terme de régularisation.

Comme cela est expliqué dans le cours, on va ramener le problème de regression polynomiale univarié à un problème de régression linéaire multivarié, en constituant une matrice X de la façon suivante à partir du vecteur colonne x de la donnée initiale :

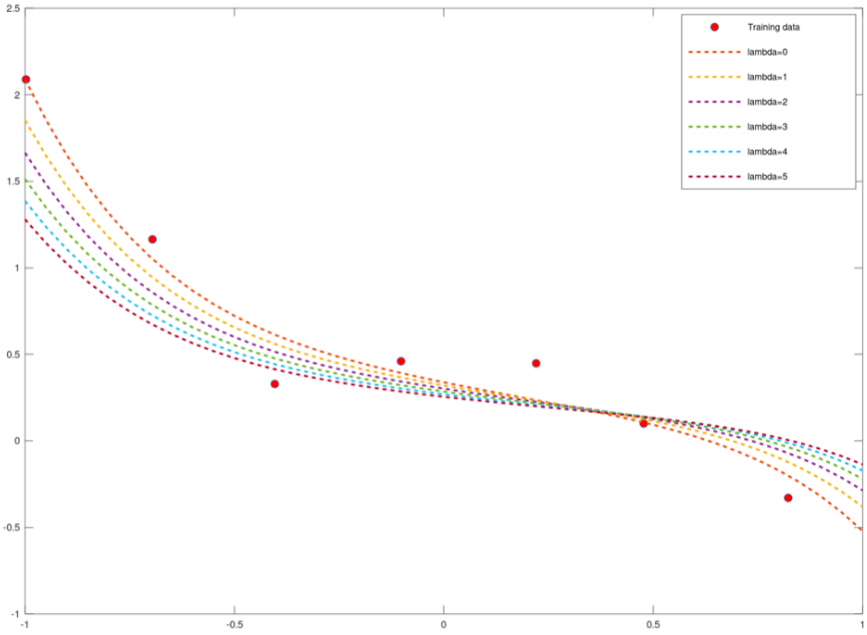
```
X=np.block([[xone,x,x**2,x**3,x**4,x**5]])
```

Réaliser cette régression pour $\lambda = 0, \lambda = 1, \lambda = 10$, puis tracer la courbe résultante sur le même graphique que les données. On doit alors obtenir une figure comparable à la suivante :



Compromis biais-variance

Calculer une valeur de λ qui réalise le bon compromis entre biais du modèle et variance du modèle. On pourra faire les mêmes calculs que précédemment avec chaque valeur λ du vecteur de valeurs $\lambda = [0, 0.5, 1, ..., 5]$ et obtenir une série de courbes comme décrites ci-dessous :



Tracer les courbes de calcul du biais et de la variance du modèle pour chacune de ces valeurs et analyser le résultat pour conclure.

Rappel de la formule de décomposition de l'erreur décrite dans le cours :

— le biais se calcule par

$$B(h(X)) = E_S[(h - f)(X)]$$

— la variance se calcule par

$$Var(h(X)) = E_S[h^2(X)] - (E_S[h(X)])^2$$

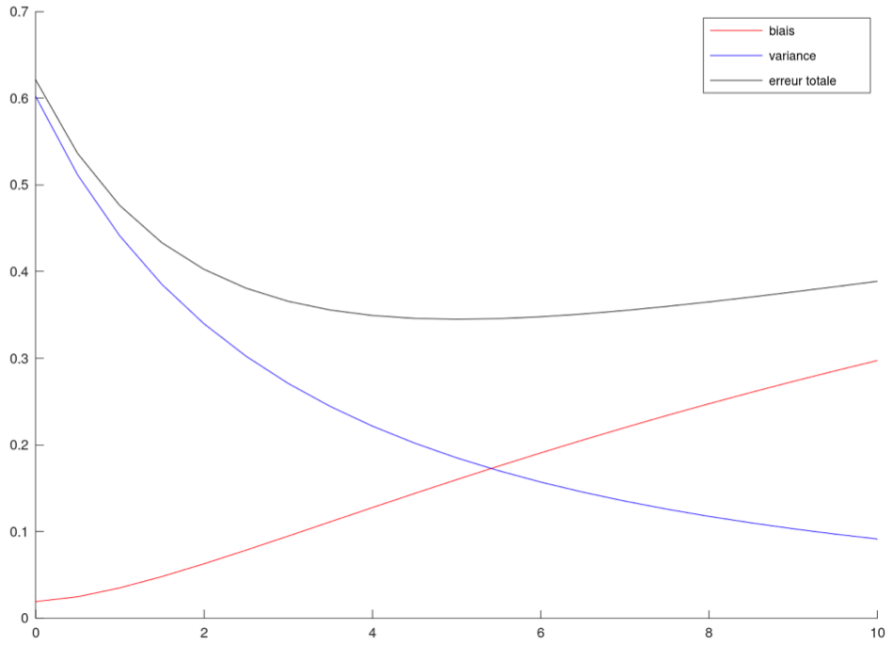
On définit ensuite l'*erreur de prédiction attendue* par

$$Err(h(X)) = B^2(h(X)) + Var(h(X)) + \sigma^2$$

où E_S correspond à l'espérance mathématique à partir d'un échantillonnage S.

L'intersection des courbes pour ces deux grandeurs en fonction de λ ainsi que l'erreur globale indiquera la valeur du compromis biais-variance.

Remarque : Comme précisé dans le cours, l'analyse biais-variance se fait à partir d'un échantillonage sur les données x à partir desquels on obtient autant de fonctions hypothèses sur lesquels on doit moyenner les résultats. Le cas présenté est donc trop réduit pour cette analyse et on en fait pas d'échantillonage. Nous allons donc donner des calculs très simplifiés du biais et de la variance dans la section suivante. Le résultat obtenu est de ce fait peu précis, mais permet quand même de fournir un degré de polynôme pertinent pour la régularisation.



Éléments de calcul du compromis biais-variance proposé

Soit X le vecteur ou la matrice des données d'entrée et Y le vecteur des données cible. Nous n'effectuons pas d'échantillonnage, ce qui revient à prendre un seul échantillon contenant toutes les données. Les calculs d'espérance se simplifient alors en de simples calculs de moyennes.

Le Biais défini par la formule

$$B(h(X)) = E_S[(h - f)(X)]$$

se simplifie ainsi, en faisant la moyenne des coefficients du vecteur $h(X) - Y$. En effet, ne connaissant pas la fonction $f(x)$, on la remplacera par les valeurs du vecteur des données cibles Y. Ce calcul s'obtient alors simplement en appelant la fonction `mean` de `numpy`, à laquelle on passera en paramètre le vecteur $h(X) - Y$.

Le calcul de la variance définie par la formule

$$Var[h(X)] = E_S[(h(X))^2] - (E_S[h(X)])^2$$

se simplifie également de manière importante et il est remplacé par un simple appel à la fonction `var` de `numpy`, à laquelle on passera en paramètre le vecteur/matrice $h(X)$.

Comme cela est mentionné, ce calcul de variance est une *interprétation* simplifiée de la définition du cours qui nécessiterait de mesurer la dispersion des valeurs de $h(x)$ par rapport à un ensemble d'échantillonages que l'on ne fait pas ici dans le TP. Ce calcul de substitution est simple mais produit quand même des résultats exploitables et semblables au comportement attendue de la *vraie* formule de variance pour les données fournies.

Les courbes du biais et de la variance sont tracées dans cet exercice en prenant en abscisses les valeurs de λ. La *complexité* du modèle est plus grande lorsque λ est petit et elle diminue lorsque λ grandit ; en effet, une grande valeur de λ va lisser fortement la courbe de régression et absorber les oscillations qu'elle pourrait produire en minimisant la fonction coût.

les sens de variation des courbes seront à l'inverse de la figure donnée en cours. Ici la courbe du biais va croitre alors que celle de la variance va décroitre.

Reprise du développement en utilisant la bibliothèque `scikit-learn`

Reprendre le programme en Python en remplaçant les fonctions de calcul de régression par les fonctions prédéfinies de la bibliothèse `scikit-learn` qui implémente les techniques de régularisation connues dont celle de Ridge sur laquelle s'appuie la méthode proposée dans le TP.